
Block Diagram for **Anti Social**

Group 2_sc_2

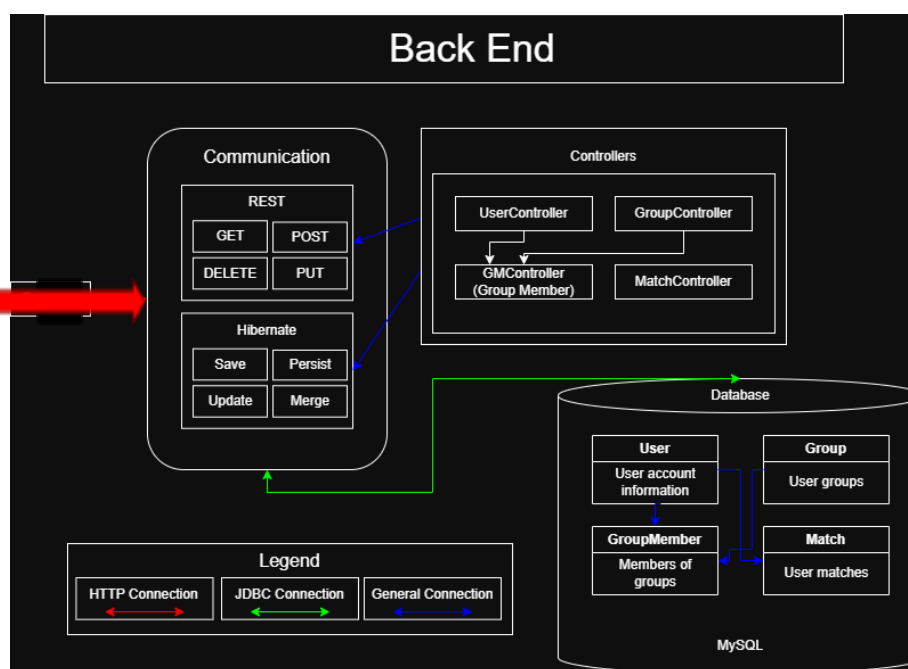
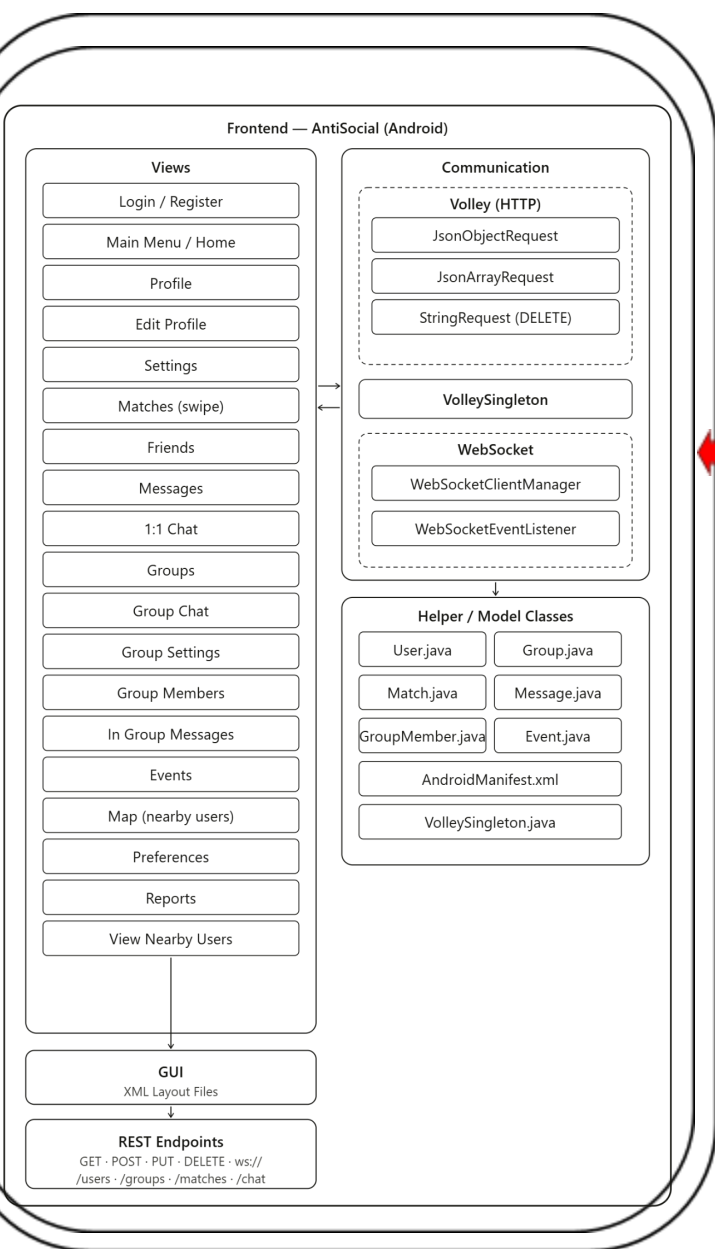
Member1 Name: Jon Buschko 25% contribution

Member2 Name: Brendan Petersen 25% contribution

Member3 Haiqa Nasir: 25%% contribution

Member4 Jongwookim: 25% contribution

BLOCK DIAGRAM



Block Diagram Description(Frontend + Backend)

Our AntiSocial application is built using a client-server architecture where the Android frontend communicates with a backend server through REST APIs and WebSocket connections. We structured the system so that UI logic, communication logic, and database logic are clearly separated.

On the **frontend side**, the application is organized into multiple activities such as Login/Register, Profile, Matches (swipe), Friends, Messaging (1:1 and Group Chat), Groups, Events, Map (nearby users), Preferences, and Reports. Each activity is responsible for displaying UI elements (defined in XML layout files) and collecting user input. The activities do not directly handle database logic; instead, they send requests to the backend when needed. This keeps the UI layer simple and easier to maintain.

For communication, we use two approaches. First, REST APIs are implemented using the Volley library. We use `JsonObjectRequest`, `JsonArrayRequest`, and `StringRequest` depending on the type of data being sent or received. All HTTP requests are managed through a single `VolleySingleton` class so that the entire app shares one request queue. This avoids creating multiple network queues and keeps the networking behavior consistent across activities.

Second, for real-time features such as 1:1 Chat and Group Chat, we use WebSocket communication. Instead of repeatedly sending HTTP requests, WebSocket keeps a persistent connection between the client and the server. This allows messages to be delivered instantly and updates the UI in real time. We separated WebSocket handling into dedicated manager and listener classes to keep the activities clean and focused only on UI behavior.

On the **backend side**, the server is structured into REST endpoints, controllers, and database layers. The REST layer handles GET, POST, PUT, and DELETE requests from the frontend. Each request is routed to a specific controller such as `UserController`, `GroupController`, `GMController` (Group Member management), and `MatchController`. These controllers process the request, apply business logic, and interact with the database.

Database interaction is handled using Hibernate ORM, which maps Java objects to MySQL tables such as `User`, `Group`, `GroupMember`, and `Match`. Instead of writing raw SQL queries in every controller, Hibernate manages operations like saving, updating, and retrieving records. This makes the code cleaner and easier to extend. The database schema supports relationships like users joining multiple groups and users matching with multiple other users.

Overall, we designed the system to be modular and easy to expand. The frontend focuses on user interaction, the backend handles logic and persistence, and real-time communication is separated from normal REST operations. This structure helps us maintain the system more easily as new features are added.

Database Schema

